Bybit Options-Income Quant-Coach

Design & Knowledge Compendium

Concepts, models, architecture, and lessons behind a private options-income decision-support system.
Ideas only — no source code.

Compiled 2026

00 — Overview

The problem being solved

Selling options premium on crypto is attractive (high implied volatility, 24/7 markets, weekly and daily expiries) but operationally punishing for a solo trader:

1. **The data is scattered.** Implied-vol surface, skew, term structure, funding, realized vol, open-interest/gamma positioning, and your own live greeks live in five different places. Deciding whether vol is *rich* right now means assembling all of it by hand.
2. **The exchange hides the true cost.** Bybit's option *mark* price is theoretical; the price you can actually transact is the bid/ask. Fees (trading + delivery) quietly eat a double-digit percentage of gross P&L. A position that looks green on mark can be a loss to close.
3. **Discipline decays.** Without a memory of what has actually worked *for you*, every decision restarts from intuition. The edge in premium selling is small and statistical; it only shows up if you measure your own record honestly.
4. **Naked short premium is unforgiving.** The book this was built for runs fully naked

short strangles on BTC — an uncapped-tail posture. That demands a risk lens that never lets a "looks fine on mark" reading hide a tail that is quietly growing.

The system exists to collapse all of that into a handful of screens that answer concrete questions: *Is vol cheap or rich? What structure fits this regime? Which of my legs needs attention? Can I actually get out of this position, and at what cost? Where is my real edge?*

Product philosophy

- **Coach, not autopilot.** The software's job is synthesis, judgment support, and memory. The human keeps the executing hand. This is a deliberate safety and skill-building choice, not a technical limitation.
- **Transparency over black boxes.** Every score is a sum of visible components, each with a reason string that carries its own point contribution. The trader can always see *why* a strategy ranked where it did. No fitted ML weights that can overfit or can't be audited.
- **Honesty about sample size.** Any statistical claim (percentile ranks, personal win-rates, seasonality) is withheld or explicitly hedged until enough data exists to support it. "Only N days tracked so far" is a first-class output.
- **First-party data where it matters.** Rather than scraping third-party visualizers, the system computes its own IV surface from the exchange and accrues its own history. External venues (Deribit) are used only as a labeled *reference*, never as a hidden signal source.
- **The flywheel.** Every trade entered snapshots the full market context at entry. That record cannot be backfilled, so capture starts on day one — and the longer the system runs, the more its calibration is worth.

The non-negotiable constraints (guardrails)

These held for the entire life of the project and shaped every feature:

1. **No order automation.** The read-only API surface is used deliberately: positions and wallet balance only. No trade, transfer, or withdrawal endpoints are ever touched.
2. **Not personalized financial advice.** Output is framed as data synthesis plus the trader's own rules. A disclaimer is always present.
3. **Per-account isolation.** The tool supports multiple sub-accounts with separate data and separate keys; no calculation ever leaks one account's data into another.
4. **Never emit invalid numbers.** `inf`/`nan` must never reach the JSON layer (they break the browser's parser). Undefined ratios are `null`, not infinity.
5. **Clearly flag estimated or missing data.** Anything unverified, placeholder, or soft-failed is labeled as such in the surface that shows it.

Technology at a glance

- **Backend:** Python, an async web framework, an async HTTP client. Pure-function core modules (all the math) separated from the I/O layer so everything is unit-testable without a network.
- **Frontend:** a single self-contained HTML file, vanilla JavaScript, no build step. Tabs for each surface; a multi-account switcher.
- **Storage:** flat JSON files on disk (per-account portfolios and closed trades; global market-history stores). No database.
- **Data sources:** Bybit public V5 (instruments, tickers, candles, linear/perp index & funding), Bybit private V5 read-only (positions, wallet balance), Deribit public v2 (DVOL index + option book summary) as an external reference.

01 — Design

This document is the conceptual core: the architecture and the full library of quant models the system is built on. Formulas are given because they *are* the ideas; no source code is included.

1. Architecture

1.1 Two layers, one machine

- **A pure-function core.** Every piece of math — fees, margin, volatility estimators, payoff geometry, scoring, calibration, backtests — lives in modules that take plain numbers and return plain numbers. No network, no disk, no global state. This is what makes the whole system unit-testable against synthetic fixtures with zero I/O.
- **A thin I/O shell.** A small async web layer fetches exchange data, reads/writes the JSON stores, and calls into the pure core. Endpoints are deliberately dumb: fetch, compute, serialize.

The discipline of "math is pure, I/O is a shell" is the single most important structural decision. It is why every model below could be verified in isolation.

1.2 Frontend

- A **single self-contained HTML file** with vanilla JavaScript and no build step. Chosen for zero-friction deployment and total transparency — the entire UI is one file you can read top to bottom.
- **Tabbed surfaces**, each mapping to one decision context (screener, intelligence, portfolio, coach, calculator, etc.).
- **Event delegation** on stable container elements rather than per-row handlers, so live re-renders never steal input focus mid-typing. Network writes are **debounced**.

1.3 Per-account isolation

- Multiple sub-accounts, each with its own data directory and its own API credentials.
- **The race-safety rule:** every request handler captures the active account's directory path into a local variable *before its first `await`*. Because an account switch can land between awaits, reading the "current account" lazily would let one request read account A's positions and write them under account B. Snapshotting up front closes that race. This one rule is applied to every account-scoped endpoint without exception.

1.4 Storage

- **Flat JSON files.** Per-account: the open portfolio and the closed-trade ledger. Global (market data, not account-scoped): the daily regime history and the intraday IV history.
- No database — the data volumes are small, the access patterns are simple, and flat files

are trivially inspectable and backup-able.

- **Global vs per-account is a deliberate split.** Market history is shared across all accounts because the market is the same for everyone; only positions and outcomes are private.

1.5 The background snapshot loop (the only "automation")

- A single background task wakes periodically and, for each underlying, captures a snapshot of the market context if one is due. It feeds **two stores at two cadences off one computation**: the daily regime store (once per UTC calendar day) and the intraday IV store (once per session window).
- **Idempotent capture:** writing "today's" snapshot replaces the existing row for that key rather than appending, so restarts and multiple fires never corrupt history.
- **Gaps are tolerated, never backfilled.** If the server was offline, that day is simply missing; the system never fabricates history.
- This is read-only capture. It is the *only* automated behavior in the entire system, and it touches nothing but its own history files.

2. Data sources

2.1 Bybit public (V5)

Option instruments and tickers (mark, mark-IV, bid/ask, greeks), daily candles, and linear/perpetual index price and funding history. The option chain is assembled by merging the instruments list with the tickers list into a unified contract object, filtered by days-to-expiry, and briefly cached.

2.2 Bybit private (V5), read-only

Only two endpoints: current option positions and unified wallet balance. HMAC-signed requests. **No trade, transfer, or withdrawal endpoint is ever called** — this is enforced by simply never implementing them in the client.

2.3 Deribit public (v2) — external reference only

Two unauthenticated calls: the DVOL volatility index and the option book summary. Used to answer "are we rich or cheap versus the deepest-liquidity venue?" — always labeled as an external reference, never fed silently into any signal.

2.4 The index-price gotcha

Bybit option tickers carry an `underlyingPrice` field that is **unreliable**. Spot is taken instead from the **linear perpetual's** `indexPrice`. Getting this wrong poisons every downstream calculation (OTM amount, margin, moneyness, greeks), so it is centralized.

3. The quant concept library

3.1 Options fee model

Realized P&L is computed **net of exchange fees**, because on a premium-selling book fees are not a rounding error — on the reference book they ran roughly **13% of gross P&L**.

- **Trading fee per fill** = `min(rate × index, 7% × option_price) × size`
  - taker rate ≈ 0.03%, maker ≈ 0.02%.
  - The **7%-of-premium cap** is the subtle part: for far-OTM cheap legs the percentage-of-premium term is smaller than the percentage-of-index term, so the cap binds. Ignoring it overstates the fee on exactly the legs a strangle-seller trades most.
- **Delivery (settlement) fee** = `min(deliveryRate × index, 12.5% × intrinsic) × size`, charged **only on exercise** (in-the-money at expiry); an out-of-the-money expiry is free.
  - `deliveryRate` ≈ 0.015% for BTC/ETH, ≈ 0.02% for alt underlyings.
  - **Exception:** *Daily* options incur **no** delivery fee. This single exception matters for breakeven math (below) and was re-verified against the exchange's own help page.
- A round-trip close pays trading fees on **both** the open and the close fill. Modeling only one side understates cost by half.

3.2 Initial-margin model (short options)

Per unit, for a short option:

`IM = max( MaxIMFactor × index − OTM_amount , MinIMFactor × index ) + max( avg_price , mark )`

then multiplied by size. Where:

- `OTM_amount(call) = max(0, strike − index)`, `OTM_amount(put) = max(0, index − strike)`.
- `MaxIMFactor`, `MinIMFactor` ≈ (0.10, 0.05) for BTC/ETH (verified against the exchange's worked example). Alt-coin factors are higher and live on a dynamically-rendered exchange page; where they can't be scraped, the BTC/ETH values are used as a **clearly-flagged placeholder** and the risk summary emits an "estimated" warning.
- A long option's margin is simply its premium.

Why hand-roll this: the unified account reports empty per-position IM/MM for options; the real figures only come from the account-level wallet balance. To attribute margin *per leg* (so the trader can see which legs to trim), the formula must be reconstructed. An early version used a wrong delta-proxy and was ~2x off on deep-OTM legs — the corrected OTM-scan formula above matches the exchange to the cent.

3.3 Contract-size convention

- Quantity is always stored in **underlying units** (e.g. BTC), not contract count.
- `1 contract = contract_size` underlying units: 0.01 for BTC/ETH, **1.0 for SOL** (a real gotcha — SOL's minimum order quantity is 1, not 0.01).
- Because qty is in underlying units, the margin formula is **size-agnostic** — it needs no knowledge of contract size. Only *contract count* and per-contract sizing need it. Keeping this straight prevents a whole class of 100x errors.

3.4 Volatility models

- **Realized-vol estimators** from OHLC candles: Parkinson (high-low range), Garman-Klass (adds open/close), and **Yang-Zhang** (adds overnight gaps; the primary estimator, most efficient for gappy 24/7 crypto).
- **RV forecast, HAR-inspired:** a fixed-weight blend of short/medium/long realized vol (weights 0.3 / 0.4 / 0.3 over 5 / 20 / 60 days of Yang-Zhang). The weights are **fixed and visible, not fitted** — deliberately un-optimizable so they cannot overfit.
- **Variance/volatility risk premium (VRP)** = `IV − forecast_RV`, reported both in vol points and as a ratio. This is the core "is selling premium worth it right now" gate.
- **Expected move** per expiry from the ATM straddle price, compared against the realized distribution of absolute moves over the same horizon — answers "is the market pricing more movement than tends to happen?"
- **Trend filter:** fast/slow moving-average relationship with a **dead-band** (~0.5%) plus a short lookback return. The dead-band is essential: a zero-tolerance MA comparison flips label on noise. (A test caught exactly this.)
- **Funding summary:** perpetual funding annualized from recent 8-hour rates — a positioning and carry signal.

3.5 ATM IV versus surface-average IV (a load-bearing distinction)

Averaging implied vol over *every* contract (all expiries, the whole smile) produces a number dominated by the expensive wings — it can read ~41% when the true at-the-money vol is ~33%. That surface-average is **not comparable** to any single-point reference like DVOL or another venue's ATM.

- The correct "IV" for cross-venue comparison and for headline display is **ATM IV**: the near-the-money implied vol at a reference expiry (the soonest with a few days to go, to skip the noisy 0-DTE smile).
- Mislabeling the surface-average as "ATM IV" and comparing it to an external ATM reference manufactures a phantom "we're 9 points rich" signal. **Always compare ATM-to-ATM.**

3.6 Skew (25-delta) and its sign convention

- **25-delta skew = put IV − call IV** at equal delta magnitude. **Positive = puts richer** (the normal fearful state).
- The sign convention must be applied *identically* everywhere it is consumed (labeling, digest signals, scoring). A sign bug that treated negative as put-rich labeled every fearful, put-rich market as "call skew" and scored it backwards — live and invisible until audited. The lesson: define the sign once, document it at every consumption site.
- When computing skew from a reference venue, the delta is reconstructed via Black-Scholes (all inputs are present) rather than trusted from a coarse strike grid, and the value is **withheld** when the nearest available strike isn't genuinely near 25-delta — a coarse

grid must not be allowed to lie.

3.7 IV rank and term structure

- **IV rank** = where current IV sits within its own recent history, as a true percentile.

Early versions used realized vol as a proxy denominator; the system now accrues real IV history (below) so the rank becomes a genuine self-referential percentile over time.

- **Term structure** = the shape of ATM IV across expiries (backwardation vs contango), which flips the attractiveness of calendar structures.

3.8 Executable close quote and Capture% (mark is not a price)

The most practically important insight in the whole system:

- Bybit's option **mark is theoretical (a model mid)**. It is *not* a price you can trade.
- To **close** a position you cross the spread: a short buys back at the **ask**, a long sells at the **bid**. That executable quote is what determines whether you can actually exit and at what cost.
- On deep-OTM legs the mark can be near zero (e.g. 0.02) while the only ask is a wide, one-sided quote (e.g. 5). "Slippage versus mark" therefore explodes to thousands of percent and is meaningless. Two corrected readings replace it:
  - **Crossing cost** measured as the **half-spread versus the bid-ask mid**, which never explodes; and a **one-sided flag** when only the side you must hit is quoted (no opposite quote → fill price is uncertain, use a limit).
  - **Capture%** = the fraction of the premium you actually keep if you close *now at the executable quote*: `(entry − buyback_ask) / entry` for a short (mirrored for a long).

This is the honest cousin of the mark-based "profit%"; the gap between the two columns is the *cost of reality* (stale marks plus the spread). A leg can show a healthy mark-profit yet a negative capture — meaning closing it actually realizes a loss.

- **Color follows economics, not liquidity.** The close-price color tracks Capture% (green only when you keep a majority of the credit), while spread width lives as a subordinate tag. An earlier version colored by spread width and painted losing-but-tight legs green — a genuine confusion that the redesign fixed.

3.9 Fee-adjusted breakeven (why it needs a root-finder)

The at-expiry underlying price where a leg nets exactly zero **after** the open trading fee **and** the delivery fee is not a closed-form expression. Because the delivery fee is `min(rate × index, 12.5% × intrinsic)`, *which term binds depends on the premium size* — cheap OTM legs hit the intrinsic cap, large legs hit the index term. So the breakeven is solved by **bisection against the exact fee model**, not a formula.

- A short put's fee-adjusted breakeven shifts **up** toward the strike (less room); a short call's shifts **down**; longs need a bigger move to overcome the fee drag.

- For *daily* options the delivery term is dropped entirely, so the breakeven collapses to a clean `strike − (entry − open_fee)`.

3.10 Payoff engine (piecewise-linear)

A generic engine computes **exact breakevens, max profit, max loss, and reward:risk for any combination of legs** by evaluating the piecewise-linear expiry payoff — the kinks are at the strikes, so the whole curve is determined by its value at and between strikes. Verified against the closed-form payoffs of standard structures. Cross-expiry structures (calendars) are excluded because their payoff isn't piecewise-linear at a single expiry.

3.11 Gamma positioning: GEX, max pain, zero-gamma flip

- **Max pain** = the strike minimizing total option-holder value (where the most open interest expires worthless) — a soft magnet into expiry.
- **Gamma exposure (GEX) profile** across strikes classifies each of your strikes as sitting in a dealer **long-gamma** zone (mean-reverting, pin-safe) or **short-gamma** zone (trend-accelerating). Heavy put open interest flags short-gamma; heavy call OI flags long-gamma.
- **Zero-gamma flip** = the spot level where the cumulative net-GEX profile crosses zero — the boundary between the stabilizing and accelerating regimes. Used as a summary field, a digest signal, and a scoring input.

3.12 Regime detection and memory

- Each underlying's regime is summarized from IV rank, VRP, trend, skew, put/call ratio, funding, and term structure, and condensed into a one-line **named regime banner** (e.g. "High-IV, range-bound, contango, premium-rich"), tinted by VRP.
- **Historical context via percentile rank:** today's reading is ranked against the book's own recent history (e.g. last 90 days), and a 5-day movement arrow is attached. Both are **withheld or hedged below a minimum sample** — "only N days tracked so far" instead of a percentile the data can't support.

3.13 Strategy playbook — transparent additive scoring

- Every candidate strategy starts at a **base score of 50**. Each signal component (VRP, IV rank, trend, skew, put/call ratio, funding, term structure, expected-move richness, gamma regime) adds or subtracts **visible points**, and each point contribution is carried inline in a human-readable **reason string**. Scores are clamped to a sane band.
- **Defined risk is structurally favored:** naked/undefined-risk structures carry a small fixed penalty plus a tie-break, so at equal signal edge a defined-risk structure outranks a naked one.
- The top pick is emitted, or **"Wait"** when nothing clears an action threshold — but the full ranked list is always attached so the UI can show *what was considered and why*.

- Because scoring is pure addition of labeled components, it is completely auditable. There is no fitted model and no hidden weight.

3.14 Strategy library (the structures scored)

A library of ~11 generators, each a pure function over a chain: cash-secured put, covered call, bull-put and bear-call credit spreads, iron condor, short strangle, **jade lizard** (short put + short call spread, emitted only when the credit ≥ the call-spread width so upside risk is structurally zero), **broken-wing butterfly** (credit put fly harvesting put skew), **put ratio spread**, **calendar** (the only long-vega/debit structure — it *inverts* the VRP/IV-rank/term-structure scoring because it wants cheap vol and contango, so low-IV regimes surface a calendar instead of "Wait"), and defensive structures: **put ladder** (short put + two lower longs, credit-only, so a crash turns *profitable*), **iron butterfly** (ATM pin play), and **seagull** (zero-cost-or-credit directional participation). Each is liquidity-filtered and payoff-annotated.

3.15 Personal-edge calibration (the learning loop)

The system leans its advice toward what has actually worked for the trader — bounded, transparent, and sample-gated:

- Closed trades are **attributed by the short leg**: a short put credits the put-selling families (CSP, bull-put, jade, broken-wing, put-ratio); a short call credits the call-selling families. Range structures (condor, strangle, calendar) are honestly **un-attributable** from leg data and get no nudge.
- The nudge is computed from **win-rate and expectancy**, and is **regime-aware**: it narrows to trades entered within a ±20 IV-rank band of the current regime when enough exist, else uses all history.
- It is **expectancy-gated** (a high win-rate can't rescue negative expectancy) and **hard-capped** (roughly ±6 points) so market signals still dominate. A minimum trade count gates it on entirely.
- When applied, each nudge adds its own reason line carrying the actual record (e.g. "your put-selling in similar IV: 8-2, +142 avg → personalized +6").
- **The flywheel that feeds it:** every entered leg snapshots the full market context at entry (IV rank, IV/RV, skew, term structure, put/call ratio, VRP, funding, trend, regime label, spot). This snapshot **cannot be backfilled**, so it is captured from day one and becomes more valuable the longer the system runs. Stored *market* history stays pure — the calibration feedback is applied only at the advice layer, never written back into the recorded regime data.

3.16 Carry monitor

Compares three sources of yield to decide where the income is: **dated-futures basis**

(annualized, cash-and-carry), **perpetual funding**, and the **VRP** (option selling). A rule-based verdict picks carry / vol / both / neither against explicit bars (e.g. basis ≥ 5%/yr, VRP ratio ≥ 1.05). A venue gotcha shaped this: the exchange **ignores the base-coin filter on linear tickers**, returning every contract, so results must be filtered by symbol prefix.

3.17 Backtesting methodology

- **Walk-forward with no lookahead**, enforced by a strict slicing convention: "the world as of day *t*" is only the data available up to *t*. Forward outcomes are measured on strictly later bars. This rule is the difference between an honest backtest and an accidental oracle.
- **Realized-vol studies:** HAR-blend forecast vs a naive trailing estimator (error/bias/beat-rate); trend-label to forward outcomes (the "falling-knife" guard's report card); and sigma-based strike touch/finish probabilities versus one-sided normal theory.
- **IV studies**, unblocked by a reference venue's multi-year DVOL history: the structural VRP edge (implied vs tenor-matched realized-forward) and the predictiveness of IV rank (does "sell when IV rank is high" pay?).
- **Headline empirical findings** (the reference book, BTC):
  - The **ATM vol-risk premium is razor-thin** (implied ≈ realized-forward over ~30 days) — so this book's edge is **not** ATM richness but the **OTM skew/tail it sells** (naked strangles) plus theta/gamma management.
  - **IV-rank timing does add** a couple of vol points (sell-high-IV-rank validated); the mid tier is noisy from overlapping windows and is reported as such.
  - Crypto **tails are ~2.7x Gaussian** at monthly horizons (a 2-sigma/30-day put finishes in-the-money ~6% of the time vs ~2% under normal theory) — an argument for shorter DTE or wider strikes on long-dated naked puts.

3.18 Risk sizing

- **Vol-targeted sizing:** scale position size by `min(1, target_vol / realized_vol)` with a floor, so exposure shrinks when the market gets loud. The realized-vol feed is the latest regime snapshot (disk read, no network).
- **Capital and margin limits** convert a capital figure and a max-utilization setting into a contracts-allowed cap; a result of `0` is a real "your limits don't allow this at current capital" signal, distinct from `null` (capital not configured).

3.19 Book-sync / reconciliation

On a cadence, the local book is diffed against the exchange's live positions to detect three kinds of drift: **new** legs present on the exchange but not tracked, **size-changed** legs (a partial close or add), and **closed-early** legs gone from the exchange before expiry. The trader is prompted to import/update or to record the close. The diff runs on every book

reload so any action clears or refreshes it immediately, rather than lingering until the next timer tick.

4. Security design

- **Encryption at rest.** Credentials and any bring-your-own API key are encrypted with a passphrase-derived key (scrypt over a per-value random salt → an authenticated symmetric cipher). The salt is embedded per value in a self-describing envelope, so there is no separate salt file to lose. A missing/wrong passphrase yields a clearly-reported **locked** state (configured but unusable) rather than a crash. Legacy plaintext is passed through and auto-migrated to ciphertext on the next save.
- **Optional login gate** for remote access: an HMAC-signed, HttpOnly session cookie with the signing key derived from the password; the whole app sits behind a device-level VPN, and the app password is defense-in-depth on top.
- **Secret hygiene is the real leak guard.** Credentials, the account directory, the bring-your-own-key file, and the market-history files are all excluded from version control. Encryption is defense-in-depth; the ignore rules are the primary control. (See [04 — Lessons](04-lessons-and-gotchas.md) for the incident that taught this the hard way.)

5. Risk constitution (the naked-strangle lens)

The reference book runs **fully naked short strangles on BTC** — an uncapped-tail posture. That shapes the risk view:

- Every "looks fine" reading is stress-tested against the tail, never just the mark. The executable-close and Capture% work exists precisely so a growing tail can't hide behind an optimistic mark.
- The backtest finding that crypto tails run far fatter than Gaussian is treated as a standing warning, not a curiosity — it argues directly against long-dated naked puts at wide-but-not-wide-enough strikes.
- Defined-risk structures are structurally favored in scoring, and the calibration loop can only *tilt*, never *override*, the market-signal and risk-limit gates.

02 — Implementation

How the system is built — described at the level of responsibilities, algorithms, and patterns. No source code; this is the "how it's put together" companion to the design.

1. Module map (responsibilities, not code)

The backend is ~45 small modules. The split that matters is **pure core** (math, testable offline) versus **I/O shell** (network, disk, endpoints).

Pure core — market & instrument math

Module	Responsibility
Chain builder	Merge instruments + tickers into unified contract objects; DTE filter; short cache
RV / IV stats	ATR, annualized realized vol, IV rank, IV/RV ratio
Volatility models	Parkinson / Garman-Klass / Yang-Zhang RV, HAR forecast, VRP, expected move, trend filter, funding summary
Fees	Trading-fee (with 7% premium cap) and delivery-fee model; round-trip fee helper
Risk / margin	Per-leg OTM-scan initial margin; position sizing; capital & utilization limits; vol-target scale
Payoff	Piecewise-linear breakeven / max-profit / max-loss / reward:risk for any leg set
P&L calculator	Side-aware gross minus round-trip fees; multi-leg totals; fee-adjusted breakeven via bisection

Pure core — synthesis & judgment

Module	Responsibility
Strategy generators	~11 structure builders, each producing candidate legs from a chain
Strategy engine	Runs all generators across the chain (calendars handled outside the per-expiry loop)
Playbook	Transparent additive scoring of every strategy; ranked output; "Wait" logic; gamma-flip
Idea engine	Re-runs the engine filtered to the coach's own strategy/DTE/delta band; sizes candidates
Roll engine	Same-strike vs strike-adjusted roll tables for flagged legs
Coach	Net signed book greeks; book assessment; prioritized position actions; setup annotation
Analytics	Closed-trade win-rate / expectancy / profit factor, segmented; discipline metrics
Calibration	Personal-edge nudges from closed trades, regime-aware, expectancy-gated, capped
Scenario	Reprice the book under spot/IV/time shocks (Black-Scholes); P&L, margin, breakeven breaches
Expiry focus	Per-held-expiry max-pain / GEX; gamma-zone tagging of your strikes
Carry	Dated-futures basis vs funding vs VRP; rule-based income verdict
Backtests	Walk-forward RV studies and IV-rule studies; no lookahead

I/O shell

Module	Responsibility
Public exchange client	Instruments, tickers, candles, linear index & funding
Private exchange client	HMAC-signed read-only positions & wallet balance

Module	Responsibility
Reference client	External venue's DVOL index + option book summary; delta reconstructed locally
Portfolio store	Leg CRUD; close (carries entry-context into the closed ledger); volume summary
Accounts store	Sub-account management; per-account directories; credential load/save
Regime history	Daily market snapshots; percentile / movement context
IV history	Intraday session-keyed IV snapshots; own-range percentile; seasonality
Crypto	Passphrase-based encryption at rest
Auth	Optional login gate
App / endpoints	~40 routes wiring the above together

2. Key algorithms in prose

2.1 Assembling the chain

Fetch instruments and tickers separately, key both by symbol, and merge into one contract per symbol carrying strike, expiry, type, mark, mark-IV, bid/ask, and greeks. Compute days-to-expiry from the expiry timestamp (options settle at 08:00 UTC). Filter by a DTE window and cache briefly so a burst of endpoints doesn't refetch.

2.2 Computing the intelligence snapshot

For one underlying: pull the chain and spot (from the perp index, never the option ticker's underlying field), compute ATM IV at the reference expiry, 25-delta skew, IV rank, term structure, put/call ratio, the volatility-model block (RV, forecast, VRP, expected move, trend, funding), and the gamma/max-pain/zero-flip block. Everything is assembled into one summary object that the playbook, the coach, and the snapshot loop all consume. Optional inputs (funding, the account's closed trades for calibration) are passed in so the same pure function serves market-only and personalized callers.

2.3 Scoring the playbook

Start each strategy at 50. For each signal, look up that strategy's weight for that signal and add `weight × signal_strength`, appending a reason string with the points inline. Apply the defined-risk tie-break and the optional personal-edge nudges (after signal scoring, before the clamp). Clamp, sort, and emit the top pick in the legacy recommendation shape (so existing consumers don't break) plus the full ranked list. If the top score is below the action threshold, the recommendation becomes "Wait" but the ranked list still ships.

2.4 Fee-adjusted breakeven by bisection

The net-at-expiry P&L as a function of the terminal underlying price is monotonic on each side of the strike but kinked by the delivery-fee `min(...)`. Rather than solve the kink analytically, bracket the breakeven and bisect against the *exact* fee model until the net

crosses zero. This automatically handles whichever fee term binds, and it agrees to the cent with the fee model used on real closes (same code path, different caller).

2.5 Personal-edge nudge

Attribute each closed trade to a strategy family by its short leg. Within the family, filter to a ±20 IV-rank band around the current regime if enough trades exist. Compute win-rate and average expectancy. Convert to a point nudge that is gated on positive expectancy and capped in magnitude, and attach the actual record as the reason. Below the minimum trade count, emit nothing.

2.6 Walk-forward backtest slicing

Iterate day *t* over the history. The "known world" at *t* is the slice up to and including *t*; the forward outcome is measured on strictly later bars over the study horizon. IV-rank percentiles are computed only over the trailing window of *past* data. Any accidental use of a future bar would inflate results, so the slice boundaries are the invariant the tests guard.

2.7 Background snapshot loop

Wake on a coarse interval. For each underlying, check whether a daily regime snapshot and/or an intraday IV snapshot is *due* (by UTC day and by session window). If either is due, compute the intelligence snapshot once and write whichever stores are due. Window-based due-ness (not a fixed instant) means an arbitrary wake phase never misses a session and absorbs daylight-saving shifts in the reference session boundaries.

3. API surface (shape, not signatures)

Roughly 40 endpoints, grouped:

- **Market:** chain, strategy suggestions, the intelligence snapshot, matrix, arb scan, carry, the RV and IV backtests.
- **Portfolio:** enriched legs (with live mark, greeks, executable close quote, per-leg margin), volume summary, cycles, risk summary, analytics.
- **Coach:** the fused brief, scenario stress test, expiry focus, roll analyzer, regime history, grounded natural-language Q&A.
- **Account & settings:** sub-account CRUD and switch, credential status, risk config, the bring-your-own-key config.
- **Reconciliation:** the position preview/diff and the import.

Every account-scoped endpoint snapshots the active account directory before its first await. Every endpoint that can hit the network soft-fails to a labeled "unavailable" rather than throwing, so one dead upstream never breaks a whole screen.

4. Frontend patterns

- **One file, tab-switched surfaces.** Each tab lazy-loads its data on first open and on an explicit refresh.
- **Event delegation on stable containers.** Editable tables (the calculator, the legs table) attach one handler to the container, not one per row, so a full re-render never moves the caret while the user is typing. Writes are **debounced**.
- **Render from synthetic payloads.** Render functions are driven purely by a data object, so they can be exercised in a browser with a fabricated payload — which is exactly how the UI is verified (below) without needing live positions in a particular state.
- **Grouping and collapsing for scale.** Once the book grows, flat lists become noise, so legs and the live-margin breakdown are grouped per underlying with subtotals, and dense per-position detail hides behind a toggle whose open/closed state survives auto-refresh.
- **Color encodes the decision, not the raw number.** Where a value's usefulness depends on interpretation (close-quote economics, capture, margin share), color follows the *decision* meaning, with the raw/secondary detail demoted to a tag.

5. Testing and verification

5.1 Unit tests against synthetic fixtures

Because the math core is pure, every model is tested offline:

- Volatility estimators are checked against **seeded geometric-Brownian-motion candles** with a known vol.
- Fees are checked against the exchange's own **worked examples**.
- The payoff engine is checked against **closed-form** payoffs of standard structures.
- The margin formula is checked against the exchange's **worked margin example**.
- Backtests are checked against **seeded ground truth** so the walk-forward slicing is provably lookahead-free.
- Calibration, scoring, and the IV/skew sign conventions each have targeted tests — including a regression test that would catch the skew-sign bug.
- Invariants that protect the JSON layer (never emit `inf`/`nan`) are asserted directly.

5.2 Browser-driven verification of the UI

Beyond unit tests, UI changes are verified by driving a real browser against the running app: render the affected surface (often from a **synthetic multi-asset payload** so edge cases appear deterministically), read the resulting DOM back, and assert on the actual rendered values and colors. Console and network are checked for errors. This catches the class of bug that unit tests can't — a formula that's right but wired to the wrong column, or a color rule that contradicts the number next to it.

5.3 The verification mindset

A change isn't "done" because the code looks right; it's done when the affected flow has been exercised and observed. Several of the most important fixes in the project (the surface-avg-vs-ATM IV bug, the close-quote color confusion) were found precisely because a rendered result was read back and compared against what the number *should* mean.

03 — Usage

How a trader actually drives the system. Organized as the workflow first, then each surface, then the decisions each one supports.

1. The daily workflow

1. **Open on the Portfolio.** The landing surface is the live book, not a screener — because the first question every session is "what do I already have, and does anything need attention?" Legs are grouped per asset with margin, unrealized P&L, and theta subtotals, ordered so the heaviest-margin asset is on top (the trim shortlist).
2. **Clear any sync drift.** If the local book has drifted from the exchange (a fill, a partial close, an early close), a banner surfaces it with one-click import/update or record-close. It refreshes on every reload so it reflects reality immediately.
3. **Read the regime.** The Intelligence/Coach surfaces answer "is vol rich, which way is the tape leaning, where's the gamma?" in one named regime line plus supporting signal cards with percentile and 5-day-movement context.
4. **Check the coach brief.** The fused view lists prioritized position actions (defend / near-expiry / roll-window legs) and new-setup ideas already annotated by how they fit the current book.
5. **Act, then record.** Execute manually on the exchange, then import or add the leg so the entry-context flywheel captures the market state at entry.
6. **Review the edge periodically.** The scorecard shows where your realized results actually come from — by strategy, underlying, DTE-at-entry, and IV-rank-at-entry.

2. The surfaces

2.1 Portfolio

The operational center. Per-leg it shows: side, strike, expiry with a DTE pill, quantity, entry, mark, the **executable close quote** (ask for shorts, bid for longs) with a spread / one-sided tag, **Capture%** (premium actually kept if closed now at that quote), profit% (on mark), theta/day, unrealized P&L, **per-leg margin** with its share of book margin, and probability-of-profit / breakeven distance.

Decisions it supports:

- *Which legs to trim* — sort by margin share; the biggest consumers are the shortlist.
- *Which shorts to buy back cheaply* — Capture% ≥ a take-profit threshold, close-quote color green, means locking in most of the credit is available now.
- *Which legs are stuck* — a one-sided or very wide close quote means you can't exit at market; use a limit.

- *Whether a "green on mark" leg is actually a loser to close* — Capture% negative says yes.

A **trade-volume strip** (premium transacted, contracts, notional) and a **collapsible live-margin panel** (grouped per underlying, sourced from the exchange's wallet balance) sit above/around the table.

2.2 Screener

Runs the strategy library across the chain and lists liquid candidates with net credit, breakevens, probability-of-profit, annualized yield, and the exact payoff KPIs (max profit, max risk, reward:risk). The playbook can deep-link here with the right strategy preselected.

2.3 Intelligence

The market read. IV rank, ATM IV (with surface-average demoted to a sub-value so they're never confused), 25-delta skew, term structure, VRP, expected-move-vs-realized table, trend, funding, gamma flip and max pain. A **carry monitor** compares dated-futures basis, funding, and VRP. An **external-reference card** shows the deepest-venue DVOL/ATM/skew with explicit "Bybit − reference" richness gaps — labeled as reference, not signal. **IV-history** and **backtest** panels show accrued own-venue history and the walk-forward studies.

2.4 Coach

The synthesis surface:

- **Trade brief** — regime cards, net signed book greeks, prioritized position actions, and new-setup ideas annotated by book-fit (with concrete, sized strikes).
- **Scorecard** — realized win-rate/expectancy/profit-factor segmented four ways, plus discipline metrics (premium-capture efficiency, loss/win size asymmetry), all with sample-size-guarded insights.
- **Scenario stress test** — three sliders (spot shock / IV shift / days forward); dial all three, then run, and see book P&L impact, projected margin/utilization with a warning banner, and newly-breached legs.
- **Expiry-week focus** — per-held-expiry max-pain/GEX with each of your strikes tagged long- or short-gamma.
- **Roll analyzer** — for flagged legs, a comparison of same-strike vs strike-adjusted rolls with net credit and resulting delta (a table, not one opaque "best" pick).
- **Ask the coach** — a grounded natural-language Q&A that answers only from the on-screen brief/scorecard data, with a strict "never invent numbers, advisory-only" prompt and a user-set daily question cap.

2.5 Calculator

A multi-leg P&L what-if. "Load my open book" opens an expiry-grouped picker that prefills legs from the live book (buy-back = current mark, index = live spot). Per leg you set taker/maker and optionally strike + type to get a **fee-adjusted breakeven**; per-leg and

whole-position net-after-fees update live. A "daily option" flag drops the delivery-fee term. `close = 0` models expire-worthless. It uses the *same* fee model as real closes, so the what-if agrees with reality to the cent.

2.6 Settings

Sub-account switch, API credential status (with a locked-state badge when encryption is on but the passphrase is missing), risk configuration (capital, max utilization, vol-target), and the bring-your-own-key config for the natural-language Q&A with its daily budget.

3. Auto-refresh and reconciliation

The portfolio auto-refreshes on a short default interval (adjustable, or off). Every refresh re-diffs the book against the exchange and updates the sync banner. Because the diff runs inside the same reload path as every mutation (import, add, close, expire, delete, account switch), any action clears or updates the banner immediately rather than leaving a stale alert until the next timer tick.

4. What the system will and won't do for you

Will: assemble the full market read, score structures transparently, size to your limits, show you the real (executable, fee-inclusive) cost of getting in and out, flag the legs that need attention, remember your entry context, and tell you where your realized edge actually is — hedging every claim by how much data backs it.

Won't: place, modify, or cancel any order; move funds; give you a number it can't support; or hide a growing tail behind an optimistic mark. Execution, and the judgment of whether to execute, stay with you.

04 — Lessons & gotchas

The bugs, surprises, and rules that came out of actually building and running the system.

These are the parts most worth carrying to any similar project.

1. Market-data traps

- **Mark is not a price.** An exchange's option *mark* is a theoretical mid. Position value, profit, and "can I get out?" must be computed from the executable **bid/ask**, not the mark.

This is the single most impactful correction in the project.

- **Slippage-versus-mark explodes on cheap legs.** With a near-zero mark, any "% versus mark" ratio blows up to thousands of percent and paints trivially-cheap buybacks as disasters.

Measure crossing cost as the **half-spread versus the bid-ask mid**, and flag one-sided books explicitly instead of emitting a nonsense number.

- **Average-over-the-smile is not ATM.** Averaging IV over all contracts is dominated by the wings and runs ~8 vol points hot. Comparing that to an external ATM reference manufactures a phantom edge. Always compare **ATM-to-ATM**.
- **Use the perp index for spot.** The option ticker's underlying-price field is unreliable; the linear perpetual's index price is the trustworthy spot.
- **The exchange ignores the base-coin filter on linear tickers.** A "give me BTC contracts" query returns *every* linear contract (other coins, even novelty futures). Filter by symbol prefix.
- **Unified account reports empty per-position option margin.** Real IM/MM come only from the account-level wallet balance; per-leg margin must be reconstructed from the formula.
- `positionValue` **is negative for shorts.** Take absolute value.

2. Money-math traps

- **Fees are ~13% of gross on a premium book.** Model them or your P&L is fiction. Both the open and the close fill are charged; a round trip is two trading fees.
- **The 7%-of-premium fee cap binds on cheap OTM legs** — exactly the legs a strangle-seller trades most. Omitting the cap overstates their cost.
- **Delivery fees exist, both sides, on exercise — except daily options.** OTM expiry is free. The "which term binds" `min(rate × index, 12.5% × intrinsic)` structure is why fee-adjusted breakeven needs a root-finder, not a formula.
- **Contract size is not universal.** BTC/ETH are 0.01 of the underlying per contract; SOL is 1.0. Storing quantity in *underlying units* keeps margin math size-agnostic and prevents 100x errors — but contract *counts* still need the per-underlying size.
- **Never let** `inf`/`nan` **reach JSON.** An infinite profit factor (no losses yet) must be `null`; infinity breaks the browser's JSON parse and silently blanks the screen.

3. Modeling discipline

- **Fixed, visible weights can't overfit.** The RV forecast blend uses hand-set weights on purpose. A fitted model would score better in-sample and lie out-of-sample.
- **Sign conventions must be defined once and honored everywhere.** A skew-sign bug (treating negative as put-rich when positive means put-rich) mislabeled and mis-scored every fearful market — live and invisible until audited. Document the sign at every consumption site.
- **Dead-bands beat exact comparisons on noisy signals.** A zero-tolerance moving-average crossover flips its label on microscopic noise; a small dead-band fixes it. A test caught this.
- **Walk-forward or it's an oracle.** The backtest's "world as of day *t*" must exclude every future bar, including in the percentile windows. The slice boundary is the invariant to test.
- **Withhold statistics you can't support.** Percentiles, seasonality marks, and personal edge are all gated on a minimum sample and say "only N so far" below it. Honesty about sample size is a feature, not a caveat.
- **Let calibration tilt, never override.** Personal-edge nudges are expectancy-gated and hard-capped so the market signal and risk limits always dominate. A learning loop that can overpower its guardrails is a liability.

4. Empirical findings worth remembering

- **The ATM vol-risk premium is razor-thin.** Over ~30-day horizons, implied ≈ realized-forward. A naked-strangle book's edge is therefore **not** ATM richness — it is the OTM **skew/tail** it sells, plus theta/gamma management. This reframed where the book thinks its edge comes from.
- **"Sell when IV rank is high" is validated** (a couple of vol points of edge), while the mid tier is noisy from overlapping-window clustering — and that noise is reported, not hidden.
- **Crypto tails run ~2.7x Gaussian at monthly horizons.** A 2-sigma/30-day put finishes ITM ~6% of the time versus ~2% under normal theory — a direct argument against wide-but-not-wide-enough long-dated naked puts.

5. Product & UX lessons

- **Color the decision, not the number.** When a value's meaning depends on interpretation, make the color track the *decision* (is this a good exit?) and demote the raw detail (spread width) to a tag. Coloring by the wrong dimension made losing-but-tight legs look green.
- **Land on the operational surface.** The first screen should be the live book, because the first question is always "what do I hold and what needs attention?"

- **Group and collapse once it scales.** Flat lists that were fine at five legs become noise at forty. Group per asset with subtotals; hide dense detail behind a toggle whose state survives refresh.
- **Reconcile on every mutation, not just on a timer.** Running the exchange-diff inside the same reload path as every action means an alert clears the instant you act on it, instead of nagging until the next tick.
- **Explicit apply beats debounced auto-run for expensive, deliberate actions.** The scenario stress test dials in three inputs and then runs on click — mid-drag auto-runs were annoying and wasteful.
- **Remove redundant alerting.** A separate notification badge was built and then deleted once the trade brief already surfaced the same priority items. Duplicated nudges erode trust.

6. Engineering & operational lessons

- **Pure core, I/O shell.** Keeping all math in pure functions is what made every model testable offline against synthetic fixtures. It is the structural decision everything else leaned on.
- **Snapshot the account directory before the first await.** Otherwise an account switch mid-request can read one account and write another. This race is silent and data-corrupting.
- **Soft-fail every network dependency.** A dead upstream returns a labeled "unavailable," not an exception, so one outage never blanks a whole screen.
- **Verify by observing the real flow.** Unit tests can't catch a correct formula wired to the wrong column. Drive the UI, read the DOM back, and compare against what the number *should* mean. The most important bugs were found this way.
- `.gitignore` **has no inline comments.** A `pattern # comment` line treats the whole thing as the pattern and silently ignores nothing — which once let plaintext credentials get tracked. Comments go on their own line, and the ignore rules are the real secret-leak guard.
- `git add -A` **does not un-track already-tracked files** even after you ignore them; they must be explicitly removed from the index. Always scan the staged diff for secrets before a commit that could ever go public.
- **Encryption at rest is defense-in-depth, not the primary control.** The primary control is never committing the secret in the first place.

7. The one-line summary

Build the boring, honest plumbing first — executable prices, real fees, correct margin, lookahead-free backtests, sample-gated statistics — and *then* the synthesis on top is trustworthy. A pretty score on top of a fictional mark is worse than no score at all.